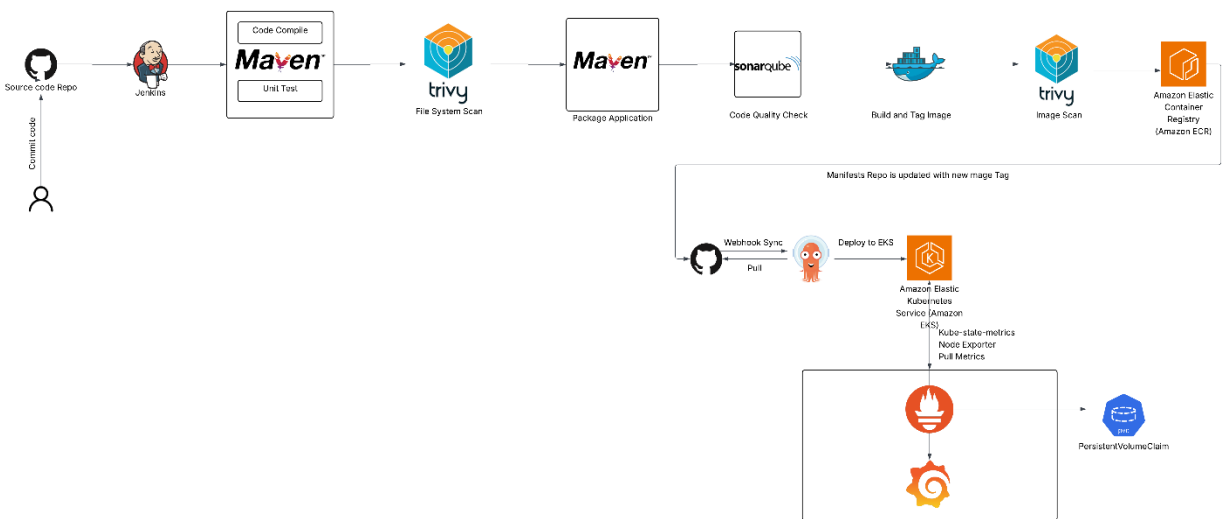


Architecture Overview

This platform implements a complete DevSecOps CI/CD workflow where Jenkins performs continuous integration tasks including build, testing, security scanning, and container image creation.

The container images are stored in Amazon Elastic Container Registry (ECR), and deployment is handled through a GitOps model using ArgoCD, which synchronizes Kubernetes manifests to an Amazon EKS cluster.

Monitoring and observability are implemented using Prometheus and Grafana to provide metrics collection and visualization for the deployed workloads.



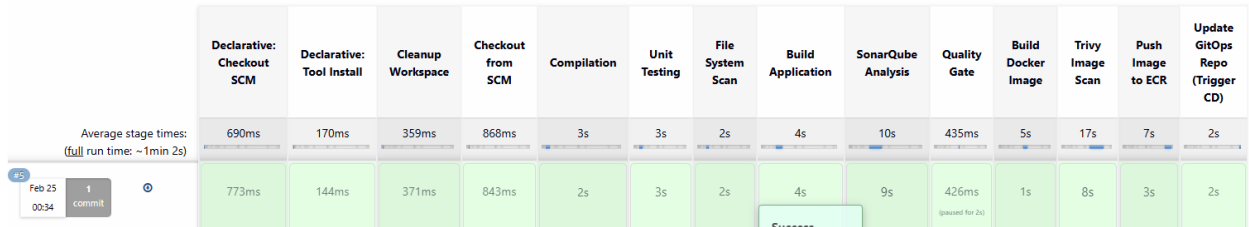
CI/CD Workflow

The CI/CD pipeline automates the complete application delivery lifecycle:

1. Developer pushes source code to the GitHub repository
2. Jenkins pipeline is triggered automatically
3. Maven compiles and tests the application
4. Trivy performs filesystem vulnerability scanning
5. SonarQube performs static code analysis
6. Docker image is built and tagged
7. Trivy scans the container image for vulnerabilities
8. The container image is pushed to AWS ECR
9. Jenkins updates the GitOps repository with the new image tag

10. ArgoCD synchronizes Kubernetes manifests to the EKS cluster
11. The application is deployed to Kubernetes
12. Prometheus collects cluster metrics and Grafana displays monitoring dashboards

Stage View



GitOps Deployment Model

The deployment strategy follows GitOps principles where the Kubernetes manifests are stored in a Git repository.

Jenkins updates the manifest with the new container image version, and ArgoCD continuously monitors the repository and synchronizes the desired state with the Kubernetes cluster.

This ensures:

- Declarative infrastructure management
- Version-controlled deployments
- Automated reconciliation of cluster state

Infrastructure Setup

Component	Instance Type	Purpose
Jenkins Master	t2.medium	CI/CD orchestrator
Jenkins Slave	t2.medium	Build agent
SonarQube	t2.medium	Code analysis
Bootstrap Server	t2.micro	EKS provisioning

Jenkins Architecture

The CI pipeline uses a Jenkins master-agent architecture.

The Jenkins master handles pipeline orchestration, while the Jenkins agent executes build workloads such as Maven builds, Docker image creation, and security scanning.

This design improves scalability and isolates build workloads from the Jenkins controller.

Jenkins Installation

Instances (2) Info				
Find Instance by attribute or tag (case-sensitive)			All states ▾	
☐	Name 🔗	Instance ID	Instance state ▾	Instance type ▾
☐	jenkins-slave	i-0d7ad8a003a3817f1	Running 🔍 🔍	t2.medium
☐	jenkins-master	i-0463229c897cc5742	Running 🔍 🔍	t2.medium

On master node

1- Installation of Java

```
sudo apt update  
sudo apt install fontconfig openjdk-21-jre  
java -version
```

```
ubuntu@ip-10-0-1-209:~$ java -version  
openjdk version "21.0.10" 2026-01-20  
OpenJDK Runtime Environment (build 21.0.10+7-Ubuntu-124.04)  
OpenJDK 64-Bit Server VM (build 21.0.10+7-Ubuntu-124.04, mixed mode, sharing)
```

2- Adding LTS Jenkins Repo and Installation

```
sudo wget -O /etc/apt/keyrings/jenkins-keyring.asc \
https://pkg.jenkins.io/debian-stable/jenkins.io-2026.key
echo "deb [signed-by=/etc/apt/keyrings/jenkins-keyring.asc]" \
https://pkg.jenkins.io/debian-stable binary/ | sudo tee \
/etc/apt/sources.list.d/jenkins.list > /dev/null

sudo apt update

sudo apt install jenkins
```

3- Starting and enabling the Service

```
sudo systemctl enable --now jenkins
```

```
ubuntu@ip-10-0-1-209:~$ systemctl status jenkins
● jenkins.service - Jenkins Continuous Integration Server
   Loaded: loaded (/usr/lib/systemd/system/jenkins.service; enabled; preset: enabled)
   Active: active (running) since Tue 2026-02-24 12:46:00 UTC; 16s ago
     Main PID: 5171 (java)
       Tasks: 47 (limit: 4667)
      Memory: 594.1M (peak: 608.6M)
         CPU: 15.878s
    CGroup: /system.slice/jenkins.service
            └─5171 /usr/bin/java -Djava.awt.headless=true -jar /usr/share/java/jenkins.war --webroot=/var/cache/jenkins/war --httpPort=8080

Feb 24 12:45:56 ip-10-0-1-209 jenkins[5171]: [LF]> This may also be found at: /var/lib/jenkins/secrets/initialAdminPassword
Feb 24 12:45:56 ip-10-0-1-209 jenkins[5171]: [LF]>
Feb 24 12:45:56 ip-10-0-1-209 jenkins[5171]: [LF]> *****
Feb 24 12:45:56 ip-10-0-1-209 jenkins[5171]: [LF]> *****
Feb 24 12:45:56 ip-10-0-1-209 jenkins[5171]: [LF]> *****
Feb 24 12:46:00 ip-10-0-1-209 jenkins[5171]: 2026-02-24 12:46:00.770+0000 [id=40] INFO jenkins.InitReactorRunner$1#onAttained: Completed initialization
Feb 24 12:46:00 ip-10-0-1-209 jenkins[5171]: 2026-02-24 12:46:00.787+0000 [id=30] INFO hudson.lifecycle.Lifecycle#onReady: Jenkins is fully up and running
Feb 24 12:46:00 ip-10-0-1-209 systemd[1]: Started jenkins.service - Jenkins Continuous Integration Server.
Feb 24 12:46:01 ip-10-0-1-209 jenkins[5171]: 2026-02-24 12:46:01.555+0000 [id=56] INFO h.m.DownloadService$Downloadable#load: Obtained the updated data file for hudson.ta-
Feb 24 12:46:01 ip-10-0-1-209 jenkins[5171]: 2026-02-24 12:46:01.559+0000 [id=56] INFO hudson.util.Retrier#start: Performed the action check updates server successfully a-
lines 1-20/20 (END)
```

On Agent node

1- Installation of Java

```
sudo apt update

sudo apt install -y openjdk-17-jdk

java -version
```

```
ubuntu@ip-10-0-1-144:~$ java -version
openjdk version "17.0.18" 2026-01-20
OpenJDK Runtime Environment (build 17.0.18+8-Ubuntu-124.04.1)
OpenJDK 64-Bit Server VM (build 17.0.18+8-Ubuntu-124.04.1, mixed mode, sharing)
```

2- Installation of Docker

```
sudo apt install -y docker.io
sudo systemctl enable docker
sudo systemctl start docker
```

Allow ubuntu user to run Docker:

```
sudo usermod -aG docker ubuntu
```

Apply group:

```
newgrp docker
```

```
ubuntu@ip-10-0-1-144:~$ systemctl status docker.service
● docker.service - Docker Application Container Engine
   Loaded: loaded (/usr/lib/systemd/system/docker.service; enabled; preset: enabled)
   Active: active (running) since Tue 2026-02-24 12:50:02 UTC; 32s ago
     TriggeredBy: ● docker.socket
    Docs: https://docs.docker.com
   Main PID: 4644 (dockerd)
      Tasks: 9
     Memory: 21.4M (peak: 21.6M)
        CPU: 271ms
    CGroup: /system.slice/docker.service
            └─4644 /usr/bin/dockerd -H fd:// --containerd=/run/containerd/containerd.sock

Feb 24 12:50:02 ip-10-0-1-144 dockerd[4644]: times="2026-02-24T12:50:02.202674958Z" level=info msg="Loading containers: start."
Feb 24 12:50:02 ip-10-0-1-144 dockerd[4644]: times="2026-02-24T12:50:02.498592709Z" level=info msg="Loading containers: done."
Feb 24 12:50:02 ip-10-0-1-144 dockerd[4644]: times="2026-02-24T12:50:02.512491663Z" level=info msg="Docker daemon" commit="28.2.2-0ubuntu1~24.04.1" containerd-snapshotter=false storage=
Feb 24 12:50:02 ip-10-0-1-144 dockerd[4644]: times="2026-02-24T12:50:02.512621914Z" level=info msg="Initializing buildkit"
Feb 24 12:50:02 ip-10-0-1-144 dockerd[4644]: times="2026-02-24T12:50:02.517778943Z" level=warning msg="CDI setup error /etc/cdi: failed to monitor for changes: no such file or directory"
Feb 24 12:50:02 ip-10-0-1-144 dockerd[4644]: times="2026-02-24T12:50:02.517794650Z" level=warning msg="CDI setup error /var/run/cdi: failed to monitor for changes: no such file or directory"
Feb 24 12:50:02 ip-10-0-1-144 dockerd[4644]: times="2026-02-24T12:50:02.532211579Z" level=info msg="Completed buildkit initialization"
Feb 24 12:50:02 ip-10-0-1-144 dockerd[4644]: times="2026-02-24T12:50:02.536728482Z" level=info msg="Docker daemon has completed initialization"
Feb 24 12:50:02 ip-10-0-1-144 dockerd[4644]: times="2026-02-24T12:50:02.536815921Z" level=info msg="API listen on /run/docker.sock"
Feb 24 12:50:02 ip-10-0-1-144 systemd[1]: Started docker.service - Docker Application Container Engine.

ubuntu@ip-10-0-1-144:~$
```

3- Installation of Trivy

```
sudo apt update
sudo apt install -y wget gnupg lsb-release

# Add Aqua Security repo key
wget -qO - https://aquasecurity.github.io/trivy-repo/deb/public.key | \
sudo gpg --dearmor -o /usr/share/keyrings/trivy.gpg

# Add repo
echo "deb [signed-by=/usr/share/keyrings/trivy.gpg]
https://aquasecurity.github.io/trivy-repo/deb $(lsb_release -sc) main" | \
sudo tee /etc/apt/sources.list.d/trivy.list

# Install Trivy
sudo apt update
sudo apt install -y trivy

#Create Trivy template directory:

sudo mkdir -p /usr/share/trivy/templates

#Download official templates:

sudo wget -O /usr/share/trivy/templates/html.tpl \
https://raw.githubusercontent.com/aquasecurity/trivy/main/contrib/html.tpl

sudo wget -O /usr/share/trivy/templates/junit.tpl \
https://raw.githubusercontent.com/aquasecurity/trivy/main/contrib/junit.tpl

sudo chmod -R 755 /usr/share/trivy
```

```
ubuntu@ip-10-0-1-144:~$ trivy --version
Version: 0.69.1
ubuntu@ip-10-0-1-144:~$
```

Enrollment of Slave inside the master

1- Create SSH Key

Happens on: MASTER

```
ssh-keygen -t rsa -b 4096 -f ~/.ssh/jenkins-agent
```

2- Copy Public Key to slave

happens on: MASTER

```
ssh-copy-id -i ~/.ssh/jenkins-agent.pub -o "IdentityFile=~/.ec2-key.pem" ubuntu@35.180.47.160
```

```
ubuntu@ip-10-0-1-209:~$ ssh-copy-id -i ~/.ssh/jenkins-agent.pub -o "IdentityFile=~/.ec2-key.pem" ubuntu@35.180.47.160
/usr/bin/ssh-copy-id: INFO: Source of key(s) to be installed: "/home/ubuntu/.ssh/jenkins-agent.pub"
/usr/bin/ssh-copy-id: INFO: attempting to log in with the new key(s), to filter out any that are already installed
/usr/bin/ssh-copy-id: WARNING: All keys were skipped because they already exist on the remote system.
(if you think this is a mistake, you may want to use -f option)
ubuntu@ip-10-0-1-209:~$
```

3- Create Jenkins working dir

Happens on: Slave

```
mkdir -p /home/ubuntu/jenkins
```

4- Register Jenkins Agent (SSH Node)

happens on: MASTER

```
ssh-copy-id -i ~/.ssh/jenkins-agent.pub ubuntu@AGENT_IP
```

Jenkins / Manage Jenkins / Nodes

Nodes

Add, remove, control and monitor the various nodes that Jenkins runs jobs on.

S	Name ↓	Architecture	Clock Difference	Free Disk Space	Free Swap Space	Free Temp Space	Response Time
	Built-In Node	Linux (amd64)	In sync	10.71 GiB	0 B	10.71 GiB	0ms
	Data obtained	20 min	20 min	20 min	20 min	20 min	20 min

Icon: S M L Legend

←

↻

⚠ Not secure 13.38.57.231:8080/manage/computer/new

☆

🔖

👤

⋮

💬 Chat

 Jenkins

/ Manage Jenkins

▼

/ Nodes

▼

/ New node

🔍

⚙️

👤

New node

Node name

jenkins-slave

Type

☒ Permanent Agent

Adds a plain, permanent agent to Jenkins. This is called "permanent" because Jenkins doesn't provide higher level of integration with these agents, such as dynamic provisioning. Select this type if no other agent types apply — for example such as when you are adding a physical computer, virtual machines managed outside Jenkins, etc.

Create

←

↻

⚠ Not secure 13.38.57.231:8080/manage/computer/createItem

☆

🔖

👤

⋮

💬 Chat

 Jenkins

/ Manage Jenkins

▼

/ Nodes

▼

🔍

⚙️

👤

Name ?

jenkins-slave

Description ?

Plain text Preview

Number of executors ?

2

Remote root directory ?

/home/ubuntu/jenkins

Labels ?

← ↻ ⚠ Not secure 13.38.57.231:8080/manage/computer/createItem

 Jenkins

Manage Jenkins

Nodes

Usage ?

Use this node as much as possible

Launch method ?

Launch agents via SSH

Host ?

10.0.1.144

Credentials ?

ubuntu

+ Add

Host Key Verification Strategy ?

Non verifying Verification Strategy

Advanced

Availability ?

Keep this agent online as much as possible

Save

 Jenkins

Nodes

+ New Node

Configure Monitors

Nodes

Add, remove, control and monitor the various nodes that Jenkins runs jobs on.

S	Name ↕	Architecture	Clock Difference	Free Disk Space	Free Swap Space	Free Temp Space	Response Time
	Built-In Node	Linux (amd64)	In sync	10.54 GiB	 0 B	10.54 GiB	0ms 
	jenkins-slave	Linux (amd64)	In sync	20.25 GiB	 0 B	20.25 GiB	54ms 
Data obtained		17 sec	17 sec	17 sec	17 sec	17 sec	17 sec

Icon: S M L

Legend

Plugins Installed

Updates

Available plugins

Installed plugins

Advanced settings

Download progress

Install	Name ↓	Released	Health
☒	Pipeline: Stage View 2.39 User Interface Pipeline Stage View Plugin.	1 mo 13 days ago	100
☒	Maven Integration 3.27 Build Tools This plugin provides a deep integration between Jenkins and Maven. It adds support for automatic triggers between projects depending on SNAPSHOTS as well as the automated configuration of various Jenkins publishers such as Junit.	6 mo 16 days ago	100
☒	Pipeline Maven Integration 1672.v1b_d4c0435b_20 pipelineMaven This plugin provides integration with Pipeline, configures maven environment to use within a pipeline job by calling sh mvn or bat mvn. The selected maven installation will be configured and prepended to the path.	1 day 15 hr ago	100
☒	Eclipse Temurin installer 146.v1898676a_f04e Provides an installer for the JDK tool that downloads the Eclipse Temurin™ build based upon OpenJDK from the Adoptium Working Group.	7 mo 24 days ago	96

Plugins Configurations:

Jenkins / Manage Jenkins / Tools

Search Settings User

+ Add Maven

Maven

Name

maven3

☒ Install automatically ?

Install from Apache

Version

3.9.4

+ Add Installer

+ Add Maven

Save

Apply



JDK installations

+ Add JDK

JDK

Name

java17

☒ Install automatically ?

Install from adoptium.net ?

Version ?

jdk-17.0.5+8

+ Add Installer

+ Add JDK

Save

Apply

SonarQube Integration



sonarqube

i-0a189e9128a3260aa



Running



t2.medium

1- Installation of Docker

```
sudo apt update
sudo apt install -y docker.io
sudo systemctl enable docker
sudo systemctl start docker
```

Allow ubuntu user to run Docker:

```
sudo usermod -aG docker ubuntu
```

Apply group:

```
newgrp docker
```

2- Creating SonarQube Container

```
#Create SonarQube Directories

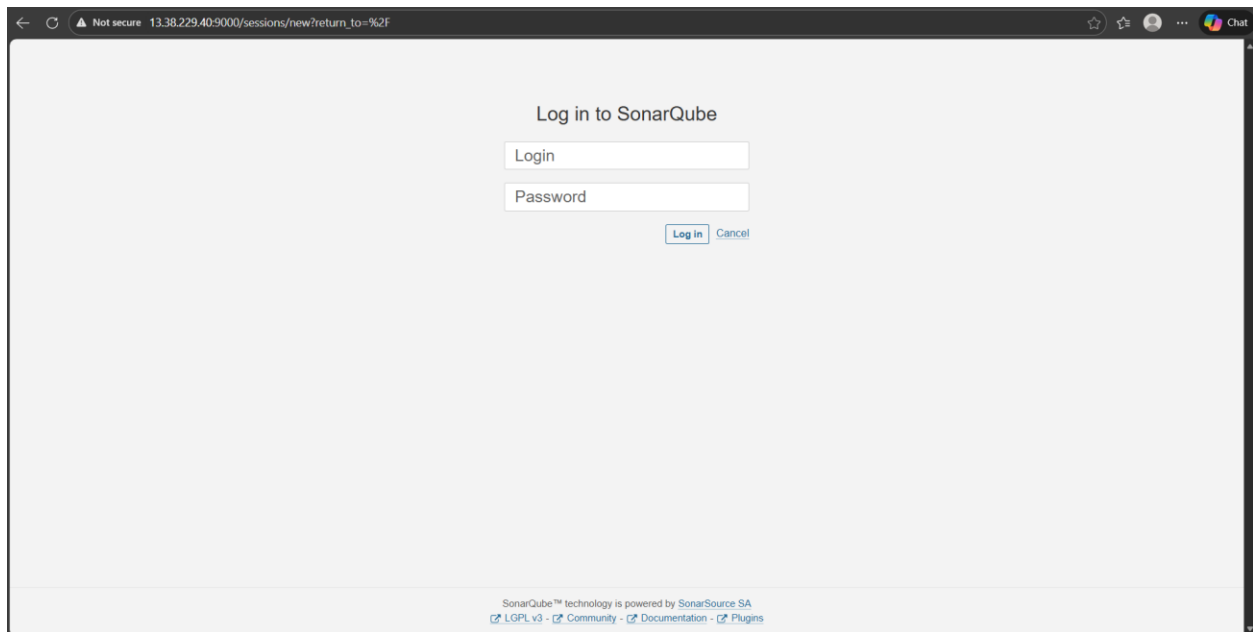
mkdir -p ~/sonarqube/{data,logs,extensions}

cd ~/sonarqube

#Run SonarQube Container

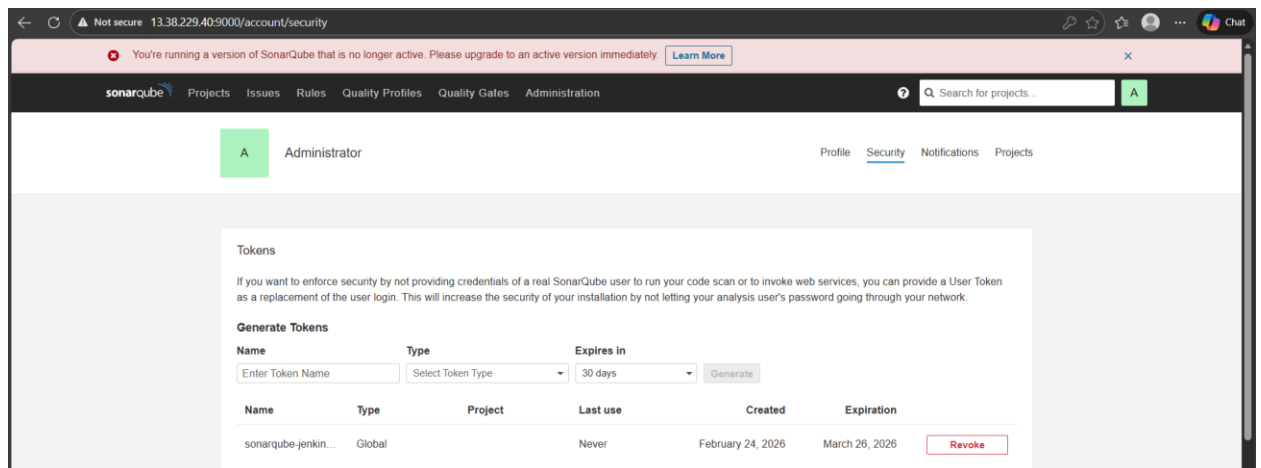
docker run -d \
--name sonarqube \
-p 9000:9000 \
-v $(pwd)/data:/opt/sonarqube/data \
-v $(pwd)/logs:/opt/sonarqube/logs \
-v $(pwd)/extensions:/opt/sonarqube/extensions \
sonarqube:lts-community
```

```
ubuntu@ip-10-0-1-109:~/sonarqube$ docker ps
CONTAINER ID   IMAGE               COMMAND                  CREATED        STATUS        PORTS                               NAMES
e54bef38343f   sonarqube:lts-community  "/opt/sonarqube/dock..."  2 minutes ago  Up 2 minutes  0.0.0.0:9000->9000/tcp, [::]:9000->9000/tcp  sonarqube
ubuntu@ip-10-0-1-109:~/sonarqube$
```

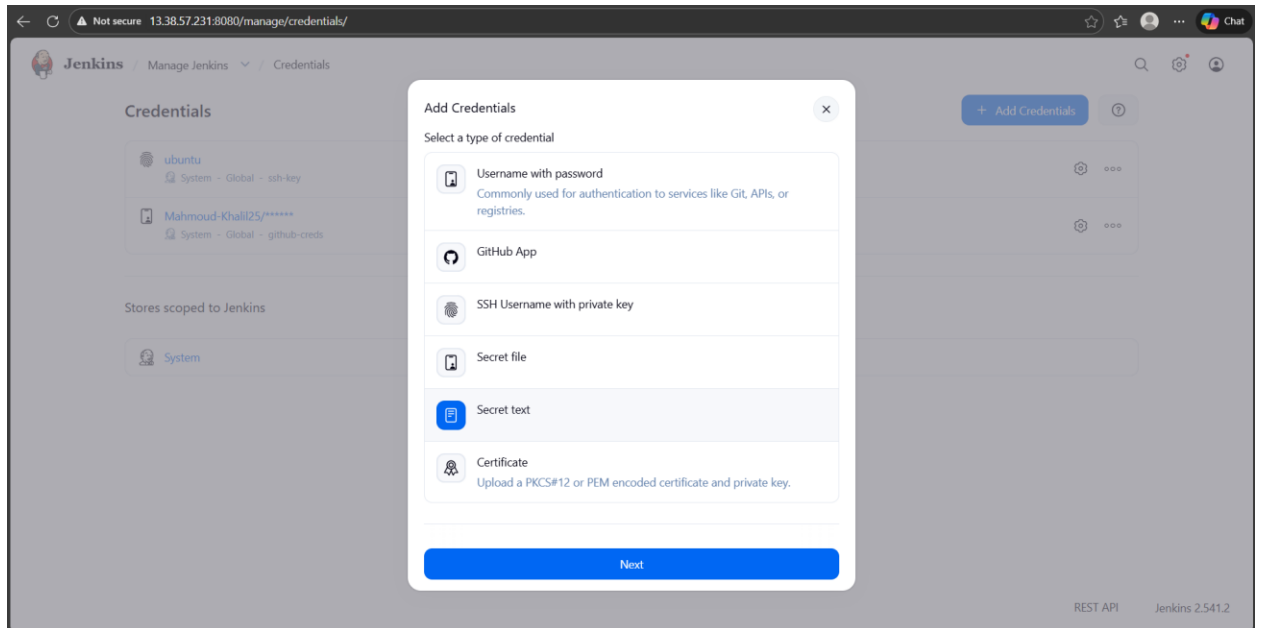


Integrating SonarQube with Jenkins

1- Generate a token for Jenkins to use in analysis



2- Create credential of the newly generated token inside Jenkins



3- Install the following plugins on jenkins

Plugins

- Updates
- Available plugins
- Installed plugins
- Advanced settings
- Download progress

sonar			
Install	Name	Released	Health
☒	SonarQube Scanner 2.18.2 External Site/Tool Integrations Build Reports This plugin allows an easy integration of SonarQube , the open source platform for Continuous Inspection of code quality.	2 mo 15 days ago	84
☒	Sonar Quality Gates 364.v67a_1255f340f Library plugins (for use by other plugins) analysis Other Post-Build Actions Fails the build whenever the Quality Gates criteria in the Sonar 5.6+ analysis aren't met (the project Quality Gates status is different than "Passed")	2 mo 9 days ago	100

4- Configure sonarqube server on Jenkins

SonarQube installations

List of SonarQube installations

Name

sonar-server

Server URL

Default is http://localhost:9000

http://10.0.1.109:9000

Server authentication token

SonarQube authentication token. Mandatory when anonymous access is disabled.

sonar-token

+

Add

Advanced

▼

Save

Apply

5- Configure sonarqube scanner on Jenkins

Jenkins

/ Manage Jenkins

/ Tools

SonarQube Scanner installations

+

Add SonarQube Scanner

Name

sonar-scanner

Install automatically

?

Version

SonarQube Scanner 8.0.1.6346

+

Add Installer

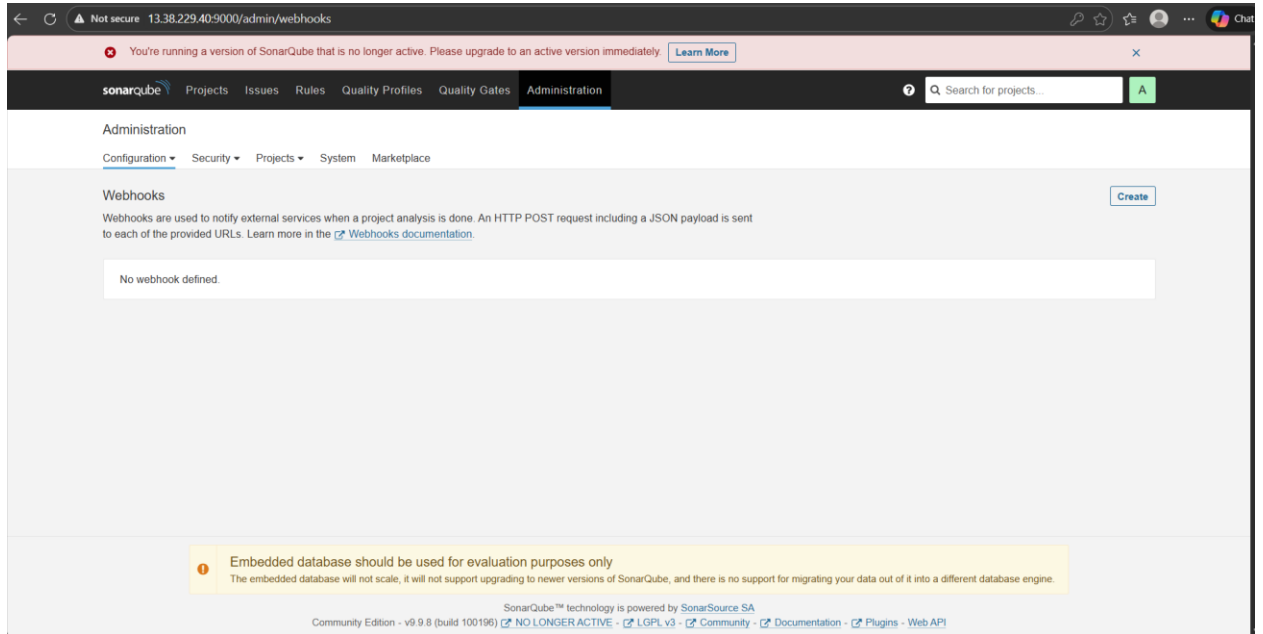
+

Add SonarQube Scanner

Save

Apply

6- Configure webhook on sonarqube



The screenshot shows the SonarQube web interface. At the top, a browser address bar displays '13.38.229.40:9000/admin/webhooks'. A red warning banner at the top of the page states: 'You're running a version of SonarQube that is no longer active. Please upgrade to an active version immediately. [Learn More](#)'. The navigation bar includes the SonarQube logo and links for Projects, Issues, Rules, Quality Profiles, Quality Gates, and Administration (which is currently selected). A search bar on the right says 'Search for projects...' with a green 'A' icon. Below the navigation bar, the 'Administration' section is expanded, showing sub-links for Configuration, Security, Projects, System, and Marketplace. The 'Webhooks' section is active, displaying a 'Create' button and a message: 'Webhooks are used to notify external services when a project analysis is done. An HTTP POST request including a JSON payload is sent to each of the provided URLs. Learn more in the [Webhooks documentation](#).' Below this, a box indicates 'No webhook defined.' At the bottom of the page, a yellow warning box states: 'Embedded database should be used for evaluation purposes only. The embedded database will not scale, it will not support upgrading to newer versions of SonarQube, and there is no support for migrating your data out of it into a different database engine.' The footer contains the text: 'SonarQube™ technology is powered by SonarSource SA. Community Edition - v9.9.8 (build 100196) [NO LONGER ACTIVE](#) - [LGPL v3](#) - [Community](#) - [Documentation](#) - [Plugins](#) - [Web API](#)'.

AWS ECR Configuration

1- Create Private Repo to push docker images

Containers

Amazon Elastic Container Registry

Share and deploy container software, publicly or privately

Amazon Elastic Container Registry (ECR) is a fully managed container registry that makes it easy to store, manage, share, and deploy your container images and artifacts anywhere.

Create a repository

Create

Pricing (US)

You pay only for the amount of data you store in your public or private repositories and data transferred to the Internet.

[ECR pricing](#)

Enable private connection to ECR private repositories using VPC interface endpoints and reduce cost.

[Learn more](#) [Deploy VPC interface endpoints](#)

How it works

```
graph LR; A[Write code<br/>Write and package code as a Docker image] --> B[Amazon ECR<br/>Compress, encrypt, and control access to images]; B --> C[Version, tag, and manage<br/>Image lifecycle]; C --> D[Run containers<br/>Pull images and run containers anywhere]; D --> E[Amazon ECS, Amazon EKS, Amazon Cloud, On premises];
```

Benefits

Fully managed

Amazon ECR eliminates the need to operate and scale the infrastructure required

Secure

Amazon ECR transfers your container images over HTTPS and automatically

Getting started

[What is Amazon ECR?](#)

[Getting started with ECR](#)

[Set up a CI/CD pipeline with ECR](#)

[Browse public gallery](#)

More Resources

Create private repository

General settings

Repository name

Enter a concise name. Repositories support namespaces, which you can use to group similar repositories.

██████████.dkr.ecr.eu-west-3.amazonaws.com/devsecops-java-app

18 out of 256 characters maximum (2 minimum). The name must start with a letter and can only contain lowercase letters, numbers, and special characters "_".

Image tag settings

Image tag mutability

Choose the tag mutability setting.

☒ Mutable
Image tags can be overwritten.

☐ Immutable
Image tags can't be overwritten.

Mutable tag exclusions

Tags that match these filters will be immutable (can't be overwritten). Using wildcards (*) will match zero or more image tag characters.

[Add Filter](#)

Filters must only contain letters, numbers, and special characters ("_", "-"). Each filter is limited to 128 characters, 2 wildcards (*), and you can add up to 5 filters in the exclusion list.

Encryption settings

Encryption configuration

The encryption settings for a repository can't be changed once the repository is created.

☒ AES-256
Industry standard Advanced Encryption Standard (AES) encryption. You can optionally choose to use a key stored in the AWS Key Management Service (KMS) to encrypt the images in your repository.

☐ AWS KMS
AWS Key Management Service (KMS)

Image scanning settings - deprecated

Successfully created private repository: devsecops-java-app

Private repositories (1)

Repository name

URI

Created at

Tag immutability

Encryption type

☐ devsecops-java-app

██████████.dkr.ecr.eu-west-3.amazonaws.com/devsecops-java-app

February 24, 2026, 17:10:50 (UTC+02)

Mutable

AES-256

[View push commands](#)

[Delete](#)

[Actions](#)

[Create repository](#)

2- Create IAM Role to allow EC2 instance to access ECR

IAM > Roles > Create role

Step 1: Select trusted entity

Select trusted entity

Trusted entity type

- ☒ **AWS service**
Allow AWS services like EC2, Lambda, or others to perform actions in this account.
- ☐ **AWS account**
Allow entities in other AWS accounts belonging to you or a 3rd party to perform actions in this account.
- ☐ **Web identity**
Allows users federated by the specified external web identity provider to assume this role to perform actions in this account.
- ☐ **SAML 2.0 federation**
Allow users federated with SAML 2.0 from a corporate directory to perform actions in this account.
- ☐ **Custom trust policy**
Create a custom trust policy to enable others to perform actions in this account.

Use case
Allow an AWS service like EC2, Lambda, or others to perform actions in this account.

Service or use case
EC2

Choose a use case for the specified service.

Use case

- ☒ **EC2**
Allows EC2 instances to call AWS services on your behalf.
- ☐ **EC2 Role for AWS Systems Manager**
Allows EC2 instances to call AWS services like CloudWatch and Systems Manager on your behalf.
- ☐ **EC2 Spot Fleet Role**
Allows EC2 Spot Fleet to request and terminate Spot instances on your behalf.
- ☐ **EC2 - Spot Fleet Auto Scaling**
Allows Auto Scaling to access and update EC2 spot fleets on your behalf.
- ☐ **EC2 - Spot Fleet Tagging**
Allows EC2 to launch spot instances and attach tags to the launched instances on your behalf.
- ☐ **EC2 - Spot Instances**

IAM > Roles > Create role

Step 1: Select trusted entity

Step 2: Add permissions

Add permissions

Permissions policies (1/1116)

Choose one or more policies to attach to your new role.

Filter by Type: All types 1 match

Policy name	Type	Description
AmazonEC2ContainerRegistryPowerUser	AWS managed	Provides full access to Amazon EC2 Container Registry

Set permissions boundary - optional

Cancel Previous Next

IAM > Roles > Create role

Step 1: Select trusted entity

Step 2: Add permissions

Step 3: Name, review, and create

Role details

Role name
Enter a meaningful name to identify this role.
JenkinsEcrPushRole

Description
Add a short explanation for this role.
Allows EC2 instances to call AWS services on your behalf.

Step 1: Select trusted entities

Trust policy

```

1 {
2   "Version": "2012-10-17",
3   "Statement": [
4     {
5       "Effect": "Allow",
6       "Action": [
7         "sts:AssumeRole"
8       ],
9       "Principal": {
10        "Service": [
11          "ec2.amazonaws.com"
12        ]
13      }
14    ]
15  }
16 }
```

Step 2: Add permissions

Permissions policy summary

3- Attach Role on EC2 Instance

Modify IAM role [Info](#)
Attach an IAM role to your instance.

Instance ID
i-0463229c897cc5742 (jenkins-master)

IAM role
Select an IAM role to attach to your instance or create a new role if you haven't created any. The role you select replaces any roles that are currently attached to your instance.

JenkinsEc2PushRole ▼ ⌚ Create new IAM role

Cancel Update IAM role

EKS Cluster Deployment

1- Install AWS CLI v2

```
sudo apt update
sudo apt install -y unzip curl

curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o "awscliv2.zip"
unzip awscliv2.zip
sudo ./aws/install
```

2- Install Kubectl

```
curl -LO "https://dl.k8s.io/release/$(curl -L -s
https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl"

chmod +x kubectl
sudo mv kubectl /usr/local/bin/
```

3- Install eksctl

```
curl -sLO "https://github.com/eksctl-
io/eksctl/releases/latest/download/eksctl_Linux_amd64.tar.gz"

tar -xzf eksctl_Linux_amd64.tar.gz
sudo mv eksctl /usr/local/bin/
```

4- Create and Attach IAM Role to Bootstrap EC2 to create EKS cluster

IAM > Roles > Create role

Step 1

Select trusted entity

Step 2

Add permissions

Step 3

Name, review, and create

Name, review, and create

Role details

Role name
Enter a meaningful name to identify this role.

ec2bootstraprole

Maximum 64 characters. Use alphanumeric and "+, -, @, _" characters.

Description
Add a short explanation for this role.

Allows EC2 instances to call AWS services on your behalf.

Maximum 1000 characters. Use letters (A-Z and a-z), numbers (0-9), tabs, new lines, or any of the following characters: _+~, @/()[]\$%&'*~:"

Step 1: Select trusted entities

Trust policy

```
1 {  
2   "Version": "2012-10-17",  
3   "Statement": [  
4     {  
5       "Effect": "Allow",  
6       "Action": [  
7         "sts:AssumeRole"  
8       ],  
9       "Principal": {  
10        "Service": [  
11          "ec2.amazonaws.com"  
12        ]  
13      }  
14    ]  
15  }  
16 }
```

Q Search

Filter by Type

All types

< 1 >

☐	Policy name	Type	Attached entities
☐	 AmazonEC2FullAccess	AWS managed	1
☐	 AmazonEKSBlockStoragePolicy	AWS managed	1
☐	 AmazonEKSClusterPolicy	AWS managed	1
☐	 AmazonEKSComputePolicy	AWS managed	1
☐	 AmazonEKSLoadBalancingPolicy	AWS managed	1
☐	 AmazonEKSNetworkingPolicy	AWS managed	1
☐	 AmazonEKSServicePolicy	AWS managed	1
☐	 AmazonEKSVPCResourceController	AWS managed	1
☐	 AmazonVPCFullAccess	AWS managed	1
☐	 AWSCloudFormationFullAccess	AWS managed	1

Customer inline 0

eks

```
1 - {
2   "Version": "2012-10-17",
3   "Statement": [
4     {
5       "Effect": "Allow",
6       "Action": [
7         "eks:DescribeClusterVersions",
8         "eks:CreateCluster",
9         "eks:DescribeCluster",
10        "eks:ListClusters",
11        "eks:TagResource",
12        "eks:UntagResource",
13        "eks:CreateNodegroup",
14        "eks:DescribeNodegroup",
15        "eks>DeleteCluster",
16        "eks:UpdateClusterConfig",
17        "eks:DescribeAddon",
18        "eks:ListAddons",
19        "eks:CreateAddon",
20        "eks:DescribeAddonVersions",
```

Copy JSON Edit

IAMFullAccess AWS managed 1

EC2 > Instances > i-0a3f882b62a1686 > Modify IAM role

Modify IAM role info

Attach an IAM role to your instance.

Instance ID i-0a3f882b62a1686 (bootstrap-server)

IAM role Select an IAM role to attach to your instance or create a new role if you haven't created any. The role you select replaces any roles that are currently attached to your instance.

ec2bootstraprole Create new IAM role

Cancel Update IAM role

5- Create EKS cluster

```
eksctl create cluster \
--name devsecops-cluster \
--region eu-west-3 \
--nodegroup-name workers \
--node-type t3a.medium \
--nodes 2 \
--nodes-min 2 \
--nodes-max 2 \
--node-volume-size 30 \
--managed
```

```

2026-02-24 21:04:47 [0] eksctl version 0.223.0
2026-02-24 21:04:47 [0] using region eu-west-3
2026-02-24 21:04:47 [0] setting availability zones to [eu-west-3c eu-west-3b eu-west-3a]
2026-02-24 21:04:47 [0] subnets for eu-west-3c - public:192.168.0.0/19 private:192.168.96.0/19
2026-02-24 21:04:47 [0] subnets for eu-west-3b - public:192.168.32.0/19 private:192.168.128.0/19
2026-02-24 21:04:47 [0] subnets for eu-west-3a - public:192.168.64.0/19 private:192.168.160.0/19
2026-02-24 21:04:47 [0] nodegroup "workers" will use "" [AmazonLinux2023/1.34]
2026-02-24 21:04:47 [!] Auto Mode will be enabled by default in an upcoming release of eksctl. This means managed node groups and managed networking add-ons will no longer be created by default. To maintain current behavior, explicitly set 'autoModeConfig.enabled: false' in your cluster configuration. Learn more: https://eksctl.io/usage/auto-mode/
2026-02-24 21:04:47 [0] using Kubernetes version 1.34
2026-02-24 21:04:47 [0] creating EKS cluster "devsecops-cluster" in "eu-west-3" region with managed nodes
2026-02-24 21:04:47 [0] will create 2 separate CloudFormation stacks for cluster itself and the initial managed nodegroup
2026-02-24 21:04:47 [0] if you encounter any issues, check CloudFormation console or try 'eksctl utils describe-stacks --region=eu-west-3 --cluster=devsecops-cluster'
2026-02-24 21:04:47 [0] Kubernetes API endpoint access will use default of {publicAccess=true, privateAccess=false} for cluster "devsecops-cluster" in "eu-west-3"
2026-02-24 21:04:47 [0] CloudWatch logging will not be enabled for cluster "devsecops-cluster" in "eu-west-3"
2026-02-24 21:04:47 [0] you can enable it with 'eksctl utils update-cluster-logging --enable-types={SPECIFY-YOUR-LOG-TYPES-HERE (e.g. all)} --region=eu-west-3 --cluster=devsecops-cluster'
2026-02-24 21:04:47 [0] default addons kube-proxy, coredns, metrics-server, vpc-cni were not specified, will install them as EKS addons
2026-02-24 21:04:47 [0]
2 sequential tasks: { create cluster control plane "devsecops-cluster",
  2 sequential sub-tasks: {
    2 sequential sub-tasks: {
      1 task: { create addons },
      wait for control plane to become ready,
    },
    create managed nodegroup "workers",
  }
}
2026-02-24 21:04:47 [0] building cluster stack "eksctl-devsecops-cluster-cluster"
2026-02-24 21:04:47 [0] deploying stack "eksctl-devsecops-cluster-cluster"
2026-02-24 21:05:17 [0] waiting for CloudFormation stack "eksctl-devsecops-cluster-cluster"
2026-02-24 21:05:48 [0] waiting for CloudFormation stack "eksctl-devsecops-cluster-cluster"
2026-02-24 21:06:48 [0] waiting for CloudFormation stack "eksctl-devsecops-cluster-cluster"
2026-02-24 21:07:48 [0] waiting for CloudFormation stack "eksctl-devsecops-cluster-cluster"
2026-02-24 21:08:48 [0] waiting for CloudFormation stack "eksctl-devsecops-cluster-cluster"
2026-02-24 21:09:48 [0] waiting for CloudFormation stack "eksctl-devsecops-cluster-cluster"
2026-02-24 21:10:48 [0] waiting for CloudFormation stack "eksctl-devsecops-cluster-cluster"
2026-02-24 21:11:48 [0] waiting for CloudFormation stack "eksctl-devsecops-cluster-cluster"

```

```

2026-02-24 21:12:49 [0] creating addon: kube-proxy
2026-02-24 21:12:49 [0] successfully created addon: kube-proxy
2026-02-24 21:12:50 [0] creating addon: coredns
2026-02-24 21:12:50 [0] successfully created addon: coredns
2026-02-24 21:12:50 [!] recommended policies were found for "vpc-cni" addon, but since OIDC is disabled on the cluster, eksctl cannot configure the requested permissions; the recommended way to provide IAM permissions for "vpc-cni" addon is via pod identity associations; after addon creation is completed, add all recommended policies to the config file, under 'addon.PodIdentityAssociations', and run 'eksctl update addon'
2026-02-24 21:12:50 [0] creating addon: vpc-cni
2026-02-24 21:12:51 [0] successfully created addon: vpc-cni
2026-02-24 21:14:51 [0] building managed nodegroup stack "eksctl-devsecops-cluster-nodegroup-workers"
2026-02-24 21:14:51 [0] deploying stack "eksctl-devsecops-cluster-nodegroup-workers"
2026-02-24 21:14:51 [0] waiting for CloudFormation stack "eksctl-devsecops-cluster-nodegroup-workers"
2026-02-24 21:15:21 [0] waiting for CloudFormation stack "eksctl-devsecops-cluster-nodegroup-workers"
2026-02-24 21:16:06 [0] waiting for CloudFormation stack "eksctl-devsecops-cluster-nodegroup-workers"
2026-02-24 21:17:57 [0] waiting for CloudFormation stack "eksctl-devsecops-cluster-nodegroup-workers"
2026-02-24 21:17:57 [0] waiting for the control plane to become ready
2026-02-24 21:17:58 [✓] saved kubeconfig as "/home/ubuntu/.kube/config"
2026-02-24 21:17:58 [0] no tasks
2026-02-24 21:17:58 [✓] all EKS cluster resources for "devsecops-cluster" have been created
2026-02-24 21:17:58 [0] nodegroup "workers" has 2 node(s)
2026-02-24 21:17:58 [0] node "ip-192-168-12-34.eu-west-3.compute.internal" is ready
2026-02-24 21:17:58 [0] node "ip-192-168-66-179.eu-west-3.compute.internal" is ready
2026-02-24 21:17:58 [0] waiting for at least 2 node(s) to become ready in "workers"
2026-02-24 21:17:58 [0] nodegroup "workers" has 2 node(s)
2026-02-24 21:17:58 [0] node "ip-192-168-12-34.eu-west-3.compute.internal" is ready
2026-02-24 21:17:58 [0] node "ip-192-168-66-179.eu-west-3.compute.internal" is ready
2026-02-24 21:17:58 [✓] created 1 managed nodegroup(s) in cluster "devsecops-cluster"
2026-02-24 21:17:58 [0] creating addon: metrics-server
2026-02-24 21:17:58 [0] successfully created addon: metrics-server
2026-02-24 21:18:00 [0] kubectl command should work with "/home/ubuntu/.kube/config", try 'kubectl get nodes'
2026-02-24 21:18:00 [✓] EKS cluster "devsecops-cluster" in "eu-west-3" region is ready
ubuntu@ip-10-0-1-12:~$

```

```

ubuntu@ip-10-0-1-12:~$ kubectl get nodes

```

NAME	STATUS	ROLES	AGE	VERSION
ip-192-168-12-34.eu-west-3.compute.internal	Ready	<none>	2m43s	v1.34.3-eks-70ce843
ip-192-168-66-179.eu-west-3.compute.internal	Ready	<none>	2m41s	v1.34.3-eks-70ce843

```

ubuntu@ip-10-0-1-12:~$

```

6- Install Helm

```

curl -fsSL https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3 |
bash
helm version

```

ArgoCD Installation

1- Create Argo CD namespace

```
kubectl create namespace argocd
```

2- Install Argo CD manifests

```
kubectl create -n argocd \
-f https://raw.githubusercontent.com/argoproj/argo-
cd/stable/manifests/install.yaml
```

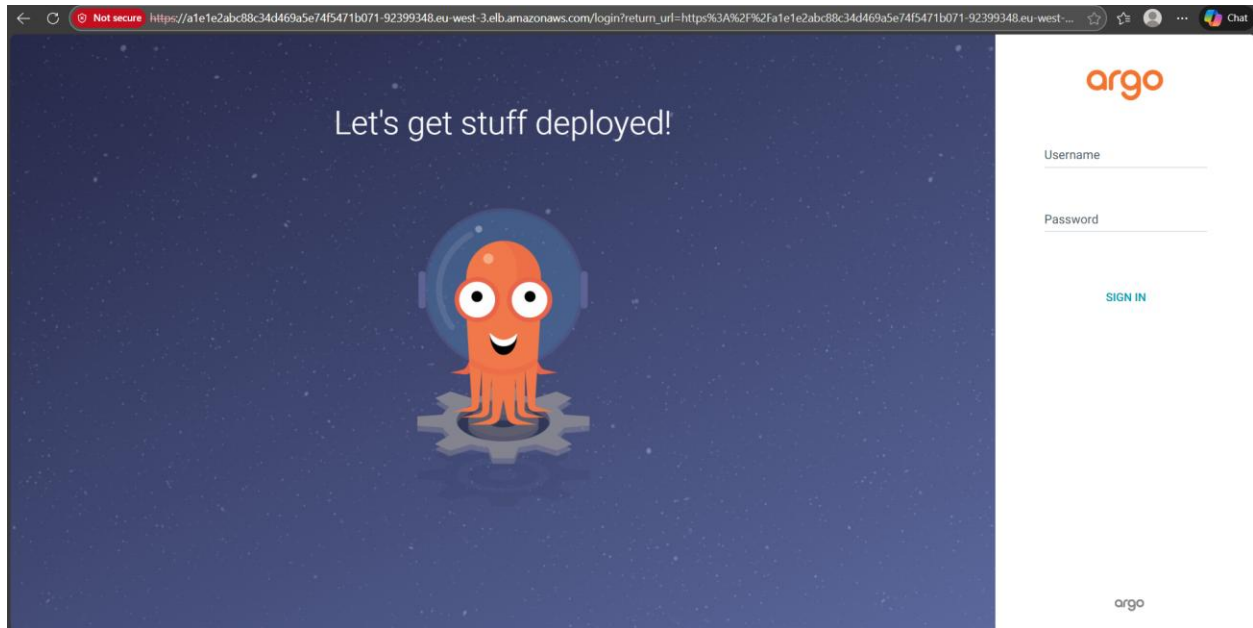
```
ubuntu@ip-10-0-1-12:~$ kubectl get pods -n argocd
NAME                                READY   STATUS    RESTARTS   AGE
argocd-application-controller-0     1/1     Running   0           34s
argocd-applicationset-controller-655bc86896-kq2mn  1/1     Running   0           34s
argocd-dex-server-74b7f44b64-k8n46  1/1     Running   0           34s
argocd-notifications-controller-7dc7577477-fvqc7  1/1     Running   0           34s
argocd-redis-796bc79b78-jkz5x       1/1     Running   0           34s
argocd-repo-server-64f7bcbf77-fqcm1  1/1     Running   0           34s
argocd-server-58fbbfdd8d-mrzgr       1/1     Running   0           34s
ubuntu@ip-10-0-1-12:~$
```

3- Expose Argo CD Externally (LoadBalancer)

```
kubectl patch svc argocd-server -n argocd \
-p '{"spec": {"type": "LoadBalancer"}}'
```

```
ubuntu@ip-10-0-1-12:~$ kubectl patch svc argocd-server -n argocd \
-p '{"spec": {"type": "LoadBalancer"}}'
service/argocd-server patched
ubuntu@ip-10-0-1-12:~$
```

```
ubuntu@ip-10-0-1-12:~$ kubectl get svc -n argocd
NAME                                TYPE                CLUSTER-IP      EXTERNAL-IP      PORT(S)                                     AGE
argocd-applicationset-controller    ClusterIP           10.100.30.131   <none>            7900/TCP,8080/TCP                          3m37s
argocd-dex-server                   ClusterIP           10.100.29.233   <none>            5556/TCP,5557/TCP,5558/TCP                 3m37s
argocd-metrics                      ClusterIP           10.100.220.152   <none>            8082/TCP                                    3m37s
argocd-notifications-controller-metrics ClusterIP           10.100.13.181   <none>            9001/TCP                                    3m37s
argocd-redis                        ClusterIP           10.100.195.6     <none>            6379/TCP                                    3m37s
argocd-repo-server                  ClusterIP           10.100.10.143    <none>            8081/TCP,8084/TCP                          3m37s
argocd-server                       LoadBalancer        10.100.109.113   a1e1e2abc88c34d469a5e74f5471b071-92399348.eu-west-3.elb.amazonaws.com  80:30388/TCP,443:32108/TCP 3m37s
argocd-server-metrics               ClusterIP           10.100.70.46     <none>            8083/TCP                                    3m37s
```

4- Install Argo CD CLI

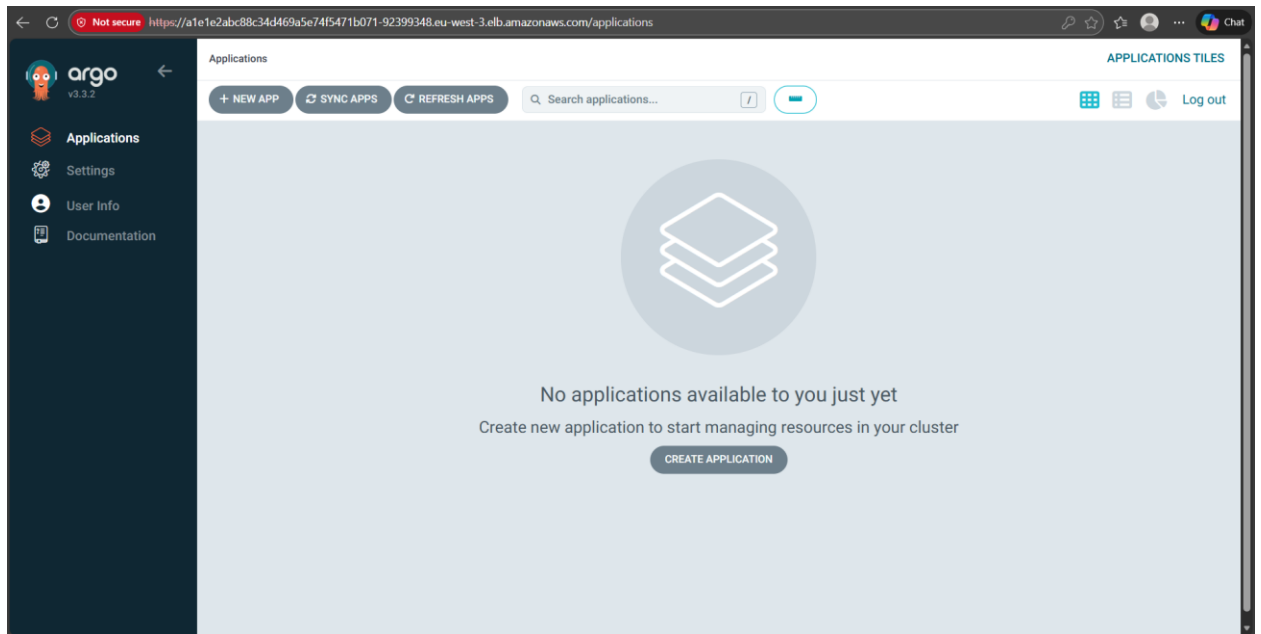
```
curl -sSL -o argocd \
https://github.com/argoproj/argo-cd/releases/latest/download/argocd-linux-amd64

chmod +x argocd
sudo mv argocd /usr/local/bin/
```

5- Retrieve and decode initial admin password

```
curl -sSL -o argocd \
https://github.com/argoproj/argo-cd/releases/latest/download/argocd-linux-amd64

chmod +x argocd
sudo mv argocd /usr/local/bin/
```



6- Add EKS cluster to Argo CD

Login to Argo CD via CLI

get password:

```
ARGOCD_PASS=$(kubectl -n argocd get secret argocd-initial-admin-secret \
-o jsonpath="{.data.password}" | base64 -d)
```

login

```
argocd login a1e1e2abc88c34d469a5e74f5471b071-92399348.eu-west-
3.elb.amazonaws.com \
--username admin \
--password $ARGOCD_PASS \
--insecure
```

Add EKS cluster to Argo CD

```
argocd cluster add $(kubectl config current-context)
```

Confirm cluster registered

```
argocd cluster list
```

```
ubuntu@ip-10-0-1-12:~$ argocd login a1e1e2abc88c34d469a5e74f5471b071-92399348.eu-west-3.elb.amazonaws.com \
--username admin \
--password $ARGOCD_PASS \
--insecure
'admin:login' logged in successfully
Context 'a1e1e2abc88c34d469a5e74f5471b071-92399348.eu-west-3.elb.amazonaws.com' updated
ubuntu@ip-10-0-1-12:~$
```

```
ubuntu@ip-10-0-1-12:~$ argocd cluster add $(kubectl config current-context)
WARNING: This will create a service account 'argocd-manager' on the cluster referenced by context 'i-0a03ff882b82a1686@devsecops-cluster.eu-west-3.eksctl.io' with full cluster level p
rivilleges. Do you want to continue [y/N]? argocd cluster list
WARNING: This will create a service account 'argocd-manager' on the cluster referenced by context 'i-0a03ff882b82a1686@devsecops-cluster.eu-west-3.eksctl.io' with full cluster level p
rivilleges. Do you want to continue [y/N]? y
{"level":"info","msg":"ServiceAccount \"argocd-manager\" created in namespace \"kube-system\"","time":"2026-02-24T21:42:08Z"}
{"level":"info","msg":"ClusterRole \"argocd-manager-role\" created","time":"2026-02-24T21:42:08Z"}
{"level":"info","msg":"ClusterRoleBinding \"argocd-manager-role-binding\" created","time":"2026-02-24T21:42:08Z"}
{"level":"info","msg":"Created bearer token secret \"argocd-manager-long-lived-token\" for ServiceAccount \"argocd-manager\"","time":"2026-02-24T21:42:08Z"}
Cluster 'https://9b235b72ba93151848912c02c36c8251.gr7.eu-west-3.eks.amazonaws.com' added
```

```
ubuntu@ip-10-0-1-12:~$ argocd cluster list
SERVER                                PROJECT                                NAME                                VERSION  STATUS  MESSAGE
https://9b235b72ba93151848912c02c36c8251.gr7.eu-west-3.eks.amazonaws.com i-0a03ff882b82a1686@devsecops-cluster.eu-west-3.eksctl.io  Unknown  Cluster has no applications and
is not being monitored.
https://kubernetes.default.svc                                             in-cluster                                             Unknown  Cluster has no applications and
is not being monitored.
ubuntu@ip-10-0-1-12:~$
```

Connect Manifests Repo to argocd

The screenshot shows the ArgoCD web interface at the URL `https://a1e1e2abc88c34d469a5e74f5471b071-92399348.eu-west-3.elb.amazonaws.com/settings/repos`. The page is titled 'Settings / Repositories' and features a sidebar with navigation links: Applications, Settings, User Info, and Documentation. The main content area displays a message: 'No repositories connected. Connect your repo to deploy apps.' There are buttons for '+ CONNECT REPO' and 'REFRESH LIST'. The page also includes a search bar and filters for Type, Project, and Status.

Not secure https://a1e1e2abc88c34d469a5e74f5471b071-92399348.eu-west-3.elb.amazonaws.com/settings/repos?addRepo=true

argo
v3.3.2

Applications
Settings
User Info
Documentation

Settings

CONNECTSAVE AS CREDENTIALS TEMPLATECANCEL

Choose your connection method:
VIA HTTP/HTTPS ▼

CONNECT REPO USING HTTP/HTTPS

Type
git ▼

Name (optional)

Project
default

Repository URL
https://github.com/Mahmoud-Khalil25/devsecops-java-eks-platform-manifests.git

Username (optional)
Mahmoud-Khalil25

argo
v3.3.2

Applications
Settings
User Info
Documentation

Settings / Repositories

REPOSITORIES

Log out

+ CONNECT REPOREFRESH LIST

Type: all ▼ Project: all ▼ Status: all ▼

Search Name

TYPE	NAME	PROJECT	REPOSITORY	CONNECTION STATUS
git		default	https://github.com/Mahmoud-Khalil25/devsecops-j...	Successful

argo
v3.3.2

←

Application

CREATE

CANCEL

×

+ NEW

Applications

Settings

User Info

Documentation

GENERAL

EDIT AS YAML

Application Name
myapp

Project Name
default

SYNC POLICY
Automatic

☒ ENABLE AUTO-SYNC ⓘ
☒ PRUNE RESOURCES ⓘ
☒ SELF HEAL ⓘ
☐ SET DELETION FINALIZER ⓘ

SYNC OPTIONS
☐ SKIP SCHEMA VALIDATION
☐ PRUNE LAST
☐ RESPECT IGNORE DIFFERENCES
☐ AUTO-CREATE NAMESPACE
☐ APPLY OUT OF SYNC ONLY
☐ SERVER-SIDE APPLY

SOURCE

Repository URL
https://github.com/Mahmoud-Khalil25/devsecops-java-eks-platform-manifests.git
GIT ✓

Revision
HEAD
Branches ▼ ⓘ

Path
./

DESTINATION

Cluster URL
https://9B235B72BA93151848912C02C36C8251.gr7.eu-west-3.eks.amazonaws.com
URL ▼

Namespace
default



v3.3.2

Applications

Settings

User Info

Documentation

Application filters

☐ ★ Favorites Only

SYNC STATUS

☐ Unknown 0

☒ Synced 1

☐ OutOfSync 0

HEALTH STATUS

☐ Progressing 0

☐ Suspended 0

☒ Healthy 1

☐ Degraded 0

☐ Missing 0

Applications

+ NEW APP

+ SYNC APPS

+ REFRESH APPS

Search applications...

7

Sort: name ▼ Items per page: 10 ▼

myapp

Project: default

Labels:

Status: Healthy Synced

Reposito... [https://github.com/Mahmoud-Khalil25/...](https://github.com/Mahmoud-Khalil25/)

Target R... HEAD

Path: /

Destinati... i-0a03ff882b82a1686@devsecops-clust...

Namesp... default

Created ... 02/25/2026 00:02:55 (a minute ago)

Last Sync: 02/25/2026 00:02:58 (a minute ago)

+ SYNC

+ REFRESH

+ DELETE

```
ubuntu@ip-10-0-1-12:~$ kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
register-app-deployment-595cf84c55-qd28m  1/1     Running   0           50s
register-app-deployment-595cf84c55-vfrfd  1/1     Running   0           50s
ubuntu@ip-10-0-1-12:~$ kubectl get svc
NAME            TYPE          CLUSTER-IP      EXTERNAL-IP      PORT(S)          AGE
kubernetes      ClusterIP     10.100.0.1       <none>            443/TCP          53m
register-app-service  LoadBalancer 10.100.46.75     a4bf1bd8c2fc04fa3a017eb3686ab3e4-839923000.eu-west-3.elb.amazonaws.com 8080:32482/TCP  70s
ubuntu@ip-10-0-1-12:~$
```



v3.3.2

Applications

Settings

User Info

Documentation

Resource filters

NAME

NAME

KINDS

KINDS

SYNC STATUS

☐ Synced 2

☐ OutOfSync 0

HEALTH STATUS

☐ Progressing 0

☐ Suspended 0

Applications / myapp

APPLICATION DETAILS TREE

DETAILS

DIFF

SYNC

SYNC STATUS

HISTORY AND ROLLBACK

DELETE

REFRESH

APP HEALTH Healthy

SYNC STATUS Synced to HEAD (e692904)

Auto sync is enabled.
Author: Mahmoud-Khalil25 <94016107+Mahmoud-Khalil25@...>
Comment: Update deployment.yaml

LAST SYNC Sync OK to e692904

Succeeded 4 minutes ago (Wed Feb 25 2026 00:02:58 GMT+0200)
Author: Mahmoud-Khalil25 <94016107+Mahmoud-Khalil25@...>
Comment: Update deployment.yaml

myapp

register-app-service

register-app-deployment

register-app-deployment-595...

register-app-deployment-595...

register-app-deployment-595...

Causing Manual Drift by changing number of replicas and disabling self heal and autosync to test the effect

```
ubuntu@ip-10-0-1-12:~$ kubectl scale deployment register-app-deployment --replicas=3
deployment.apps/register-app-deployment scaled
ubuntu@ip-10-0-1-12:~$
```

The screenshot shows the Argo CD web interface for an application named 'myapp'. The left sidebar contains navigation links for Applications, Settings, User Info, and Documentation, along with resource filters. The main panel displays the application's status: 'APP HEALTH' is 'Healthy' (green heart), 'SYNC STATUS' is 'OutOfSync' (yellow warning icon) from HEAD (e692904), and 'LAST SYNC' is 'Sync OK' (green checkmark) to e692904. Below this, a detailed sync log shows the deployment process. The central diagram illustrates the application's structure: 'myapp' (Application) is linked to 'register-app-service' (Service), which is linked to 'register-app-deployment' (Deployment). This deployment is linked to 'register-app-deployment-595...' (ReplicaSet), which is linked to three 'pod' resources. The 'register-app-deployment' resource shows a status of '31 minutes / rep:1'. The 'register-app-deployment-595...' resource shows a status of '31 minutes / rep:3'. The three 'pod' resources show a status of '31 minutes / running / 1/1 / more'.

The screenshot shows the GitHub repository page for 'devsecops-java-eks-platform-manifests'. The repository is public and has 1 branch and 0 tags. The main branch is 'main'. The repository has 3 commits. The commit history shows a commit by 'jenkins' with the message 'deploy: update image tag to 5' and a commit by 'fef2ee3' with the message 'deploy: update image tag to 5'. The commit 'fef2ee3' is the most recent and is 1 minute old.

Code Blame 23 lines (23 loc) · 506 Bytes

Raw Copy Download Edit View

```
1  apiVersion: apps/v1
2  kind: Deployment
3  metadata:
4    name: register-app-deployment
5  spec:
6    replicas: 2
7    selector:
8      matchLabels:
9        app: register-app
10   template:
11     metadata:
12       labels:
13         app: register-app
14     spec:
15       containers:
16         - name: register-app
17           image: 014887028517.dkr.ecr.eu-west-3.amazonaws.com/devsecops-java-app:5
18       resources:
19         limits:
20           memory: "256Mi"
21           cpu: "500m"
22       ports:
23         - containerPort: 8080
```


Monitoring Setup

The monitoring stack is deployed using the kube-prometheus-stack Helm chart, which provides Prometheus for metrics collection and Grafana for visualization.

This allows monitoring of Kubernetes nodes, pods, and cluster performance through preconfigured dashboards.

1- Add the Prometheus community repo

```
helm repo add prometheus-community https://prometheus-  
community.github.io/helm-charts  
helm repo update
```

```
ubuntu@ip-10-0-1-12:~$ helm repo add prometheus-community https://prometheus-community.github.io/helm-charts  
"prometheus-community" has been added to your repositories  
ubuntu@ip-10-0-1-12:~$ helm repo update  
Hang tight while we grab the latest from your chart repositories...  
...Successfully got an update from the "prometheus-community" chart repository  
Update Complete. *Happy Helming!*
```

2- Install kube-prometheus-stack with PVC + external Grafana

Create namespace

```
kubectl create ns monitoring
```

Install with PVC + LB Grafana

This installs: Prometheus, Grafana, Alertmanager, node-exporter, kube-state-metrics, dashboards.

```
helm install monitoring prometheus-community/kube-prometheus-stack \
-n monitoring \
--set grafana.service.type=LoadBalancer \
--set grafana.adminPassword='admin123' \
--set prometheus.prometheusSpec.retention=7d \
--set
prometheus.prometheusSpec.storageSpec.volumeClaimTemplate.spec.accessModes[0]=ReadWriteOnce \
--set
prometheus.prometheusSpec.storageSpec.volumeClaimTemplate.spec.resources.requests.storage=10Gi \
--set
alertmanager.alertmanagerSpec.storage.volumeClaimTemplate.spec.accessModes[0]=ReadWriteOnce \
--set
alertmanager.alertmanagerSpec.storage.volumeClaimTemplate.spec.resources.requests.storage=5Gi
```

```

ubuntu@ip-10-0-1-12:~$ kubectl create ns monitoring
namespace/monitoring created
ubuntu@ip-10-0-1-12:~$ helm install monitoring prometheus-community/kube-prometheus-stack \
-n monitoring \
--set grafana.service.type=LoadBalancer \
--set grafana.adminPassword='admin123' \
--set prometheus.prometheusSpec.retention=7d \
--set prometheus.prometheusSpec.storageSpec.volumeClaimTemplate.spec.accessModes[0]=ReadWriteOnce \
--set prometheus.prometheusSpec.storageSpec.volumeClaimTemplate.spec.resources.requests.storage=10Gi \
--set alertmanager.alertmanagerSpec.storage.volumeClaimTemplate.spec.accessModes[0]=ReadWriteOnce \
--set alertmanager.alertmanagerSpec.storage.volumeClaimTemplate.spec.resources.requests.storage=5Gi
NAME: monitoring
LAST DEPLOYED: Tue Feb 24 22:47:51 2026
NAMESPACE: monitoring
STATUS: deployed
REVISION: 1
TEST SUITE: None
NOTES:
kube-prometheus-stack has been installed. Check its status by running:
  kubectl --namespace monitoring get pods -l "release=monitoring"

Get Grafana 'admin' user password by running:

  kubectl --namespace monitoring get secrets monitoring-grafana -o jsonpath="{.data.admin-password}" | base64 -d ; echo

Access Grafana local instance:

  export POD_NAME=$(kubectl --namespace monitoring get pod -l "app.kubernetes.io/name=grafana,app.kubernetes.io/instance=monitoring" -o name)
  kubectl --namespace monitoring port-forward $POD_NAME 3000

Get your grafana admin user password by running:

  kubectl get secret --namespace monitoring -l app.kubernetes.io/component=admin-secret -o jsonpath="{.items[0].data.admin-password}" | base64 --decode ; echo

```

3- Add Grafana PVC

```

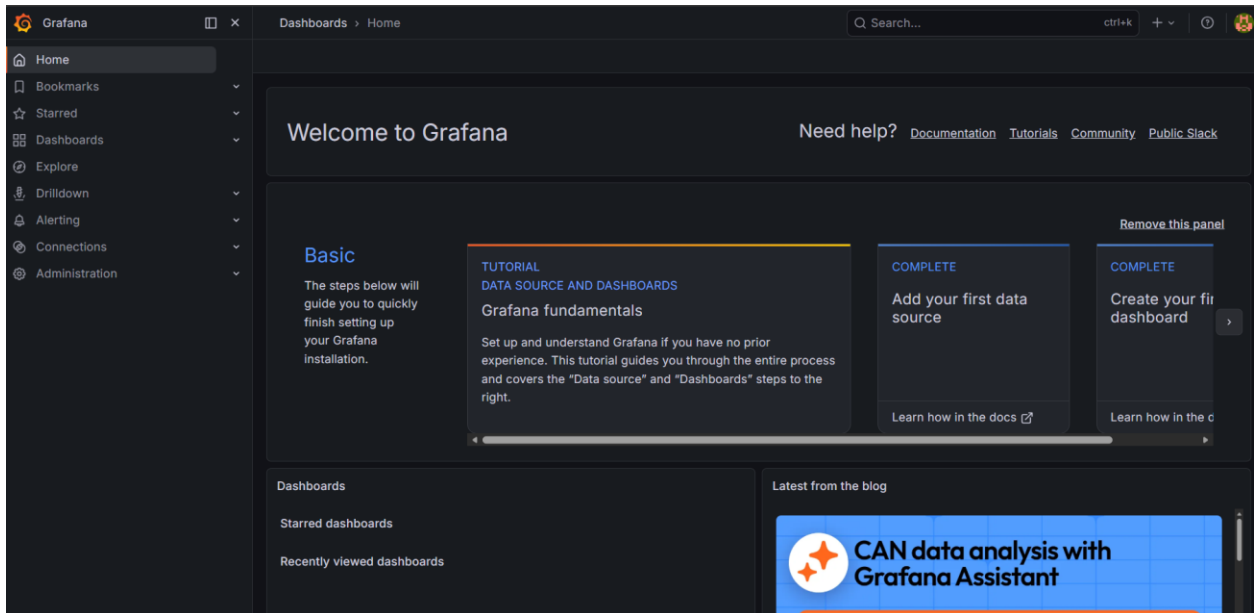
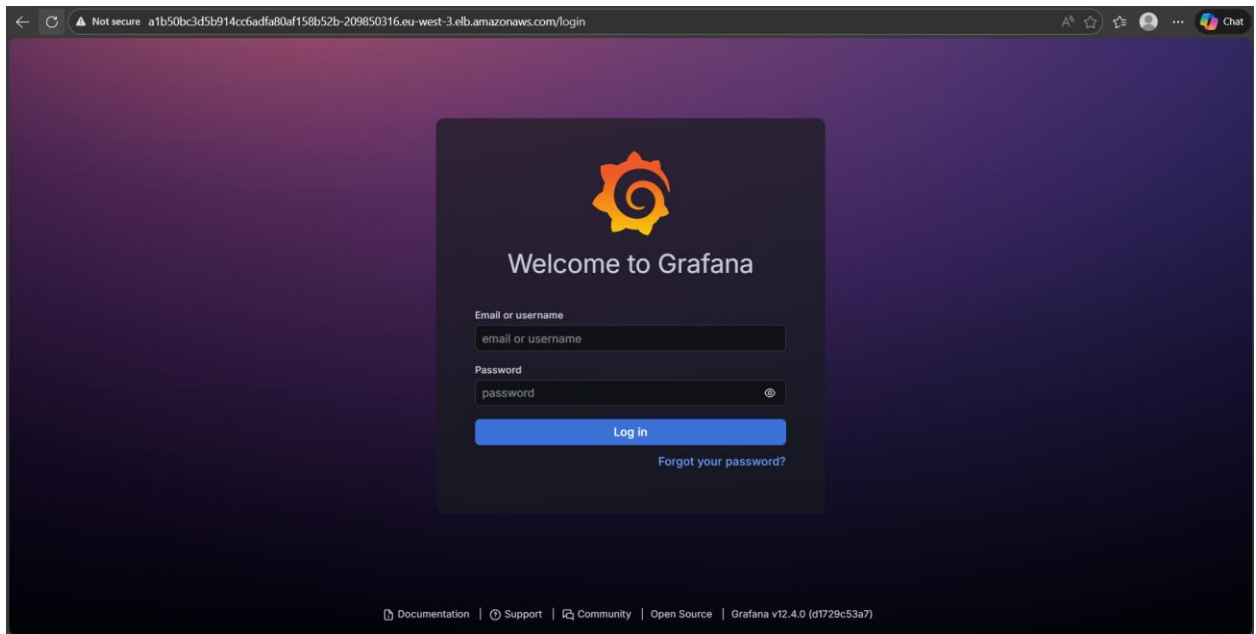
helm upgrade monitoring prometheus-community/kube-prometheus-stack \
-n monitoring \
--reuse-values \
--set grafana.persistence.enabled=true \
--set grafana.persistence.size=5Gi

```

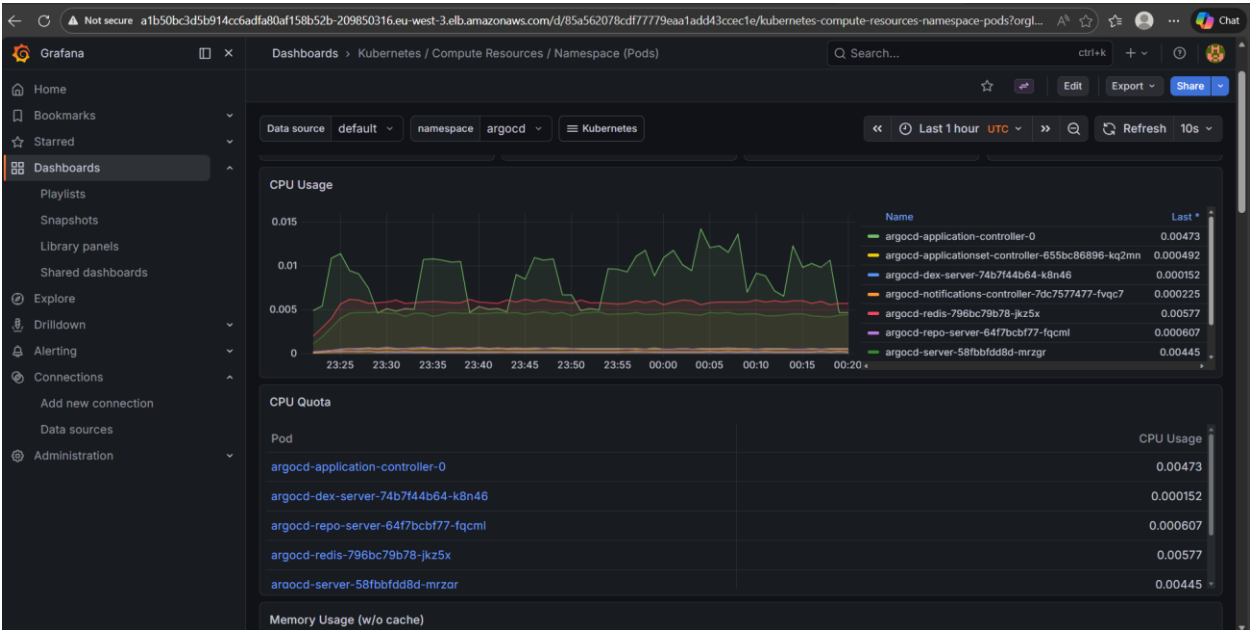
```

ubuntu@ip-10-0-1-12:~$ kubectl get pvc -n monitoring
NAME                                STATUS  VOLUME                                     CAPACITY  ACCESS
MODES  STORAGECLASS  VOLUMEATTRIBUTESCLASS  AGE
alertmanager-monitoring-kube-prometheus-alertmanager-db-alertmanager-monitoring-kube-prometheus-alertmanager-0  Bound  pvc-33b562c4-dbc4-4454-b81f-66931ac8bb9c  5Gi  RWO
gp3-csi  <unset>          53s
monitoring-grafana                  Bound  pvc-791f4578-9d46-416f-a315-2137af3a49a8  5Gi  RWO
gp3-csi  <unset>          23s
prometheus-monitoring-kube-prometheus-prometheus-db-prometheus-monitoring-kube-prometheus-prometheus-0      Bound  pvc-4daf6217-ec55-46c7-86a6-8167602b8398  10Gi  RWO
gp3-csi  <unset>          52s
ubuntu@ip-10-0-1-12:~$

```



You can see dashboards related to eks



Memory Quota

Pod	Memory Usage	Memory Usage (RSS)	Memory Usage (Cache)
argocd-application-controller-0	153 MiB	152 MiB	140 KiB
argocd-repo-server-64f7bc7f77-fqcm1	23.9 MiB	21.9 MiB	400 KiB
argocd-dex-server-74b7f44b64-k8n46	38.9 MiB	17.3 MiB	23.5 MiB
argocd-repo-server-64f7bc7f77-fqcm1	4.60 MiB	4.00 MiB	0 B
argocd-applicationset-controller-655bc86896-kq2mn	20.8 MiB	19.9 MiB	0 B

